

School Management App

Notifications Modul – Értesítések

React 18 + Laravel 12 API | 2026

1. A modul felépítése

A Notifications modul az egyszerűbb modulok közé tartozik, de két egyedi jellemzővel rendelkezik: nincs szerkesztés mód (csak létrehozás és törlés), és van egy valós idejű olvasatlan számláló a Navbar-ban.

Komponens / Fájl	Funkció
NotificationListPage	Az összes értesítés listázása (csak admin látja)
NotificationFormPage	Új értesítés küldése – nincs szerkesztés mód!
MyNotificationsPage	Bejelentkezett user saját értesítései + olvasottnak jelölés
Navbar badge	Olvasatlan értesítések száma, 60 másodpercenként frissül
useNotifications.js	React Query hook-ok: lista, my, unreadCount, create, delete, markAsRead

2. Miért nincs szerkesztés mód?

Értesítéseket nem szerkesztünk – üzleti döntés

Az értesítések küldése végleges aktus – ha egy értesítés már elküldésre került, nem illendő utólag módosítani (a felhasználók már olvashatták). Ezért a backend `Route::apiResource()->except(['update'])` mintát alkalmaz: szándékosan kivettük a `PUT /notifications/{id}` végpontot. A frontend ezt tükrözi: nincs 'Szerkesztés' gomb a listában, és a `NotificationFormPage`-nek nincs edit módja (nincs `useParams()` + `isEditMode` logika).

3. Navbar badge – refetchInterval

```
export function useUnreadCount() {
  return useQuery({
    queryKey: ['notifications', 'unread-count'],
    queryFn: () =>
      api.get('/notifications/meta/unread-count')
```

```

        .then((res) => res.data.data),
        refetchInterval: 60 * 1000, // 60 másodpercenként automatikusan
        frissül
    });
}

// Navbar-ban a helyes mezőnév:
const unreadCount = unreadData?.count ?? 0;

// Badge megjelenítése csak ha van olvasatlan:
{unreadCount > 0 && (
    <span className='badge rounded-pill bg-danger'>
        {unreadCount > 99 ? '99+' : unreadCount}
    </span>
)}

```

Miért refetchInterval és nem WebSocket?

Egy valós idejű értesítési rendszer WebSocket-et (pl. Laravel Echo + Pusher) igényelne – ez jóval összetettebb infrastruktúrát jelent. Mivel ez egy tanulási projekt és az értesítések nem kritikusán valós idejűek, a 60 másodpercenkénti polling egyszerű és hatékony kompromisszum. Ha a felhasználó olvasottak jelöl egy értesítést, az invalidateQueries azonnal frissíti a számot – nem kell megvárni a következő automatikus lekérdezést.

4. markAsRead – kettős cache invalidálás

```

export function useMarkAsRead() {
    const queryClient = useQueryClient();

    return useMutation({
        mutationFn: (id) => api.post(`/notifications/${id}/read`),
        onSuccess: () => {
            // ['notifications'] prefix-egyezéssel MINDENKIT frissít:
            // - ['notifications', page]          → lista
            // - ['notifications', 'my']           → saját értesítések
            // - ['notifications', 'unread-count'] → Navbar badge
            queryClient.invalidateQueries({ queryKey: ['notifications']
        });
    },
});

```

Miért elegendő egyetlen invalidateQueries hívás?

A ['notifications'] prefix-egyezés az összes notifications-szal kezdődő cache bejegyzést érvényteleníti egyszerre. Olvasottak jelölés után három dolog változik: 1) A saját értesítések listájában az is_read mező true lesz, 2) Az olvasatlan szám csökken a Navbar badge-en, 3) Az admin lista is frissül. Mind a három különböző queryKey-en van tárolva, de mind a három 'notifications'-szal kezdődik – egyetlen invalidateQueries kezeli mindet.

5. Route sorrend – statikus route-ok előre

```
// HELYES sorrend:
<Route path='/notifications/my'      element={<MyNotificationsPage />} />
<Route path='/notifications/create'  element={<NotificationFormPage />} />
<Route path='/notifications'         element={<NotificationListPage />} />

// Ha /notifications/:id route-ot is felvinnénk (pl. részlet oldal),
// annak a statikus route-ok UTÁN kellene szerepelnie:
// <Route path='/notifications/:id' element={<NotificationDetailPage />} />
/>
```

Miért nincs /:id route a Notifications modulban?

Mivel nincs szerkesztés oldal, és a részletek (title, message) már a listában megjelennek (a message első 60 karaktere), nem szükséges külön részlet oldal. Az egyszerűbb moduloknál ez teljesen elfogadható – nem kötelező minden modulnak ugyanolyan struktúrát követnie.

6. A visszatérő frontend/backend eltérési hiba

Frontend küldött	Backend várt	Mező
general, academic, event...	notice, announcement, reminder, alert	type
target_role: 'all'	target: 'all'	mező neve
hiányzott	channel: 'app' 'email' 'sms'	channel (kötelező!)
unreadData?.unread_count	unreadData?.count	unread badge mezőnév

Mi okozta ezeket az eltéréseket?

A frontend kód megírása előtt nem néztük meg részletesen a StoreNotificationRequest validációs szabályait – a mezőneveket és értékeket feltételezésből írtuk be. A tanulság: minden új form megírásakor ELŐSZÖR nézzük meg a Form Request rules() metódusát, és onnan másoljuk a pontos mezőneveket és megengedett értékeket. Ez megspórol egy próba-hiba kört.

7. MyNotificationsPage – kártya alapú megjelenítés

```
// Olvasott vs olvasatlan értesítés vizuális megkülönböztetése:
<div className={`card border-start border-4 ${
  notification.is_read
    ? 'border-secondary opacity-75' // halványabb, olvasott
    : 'border-primary'               // kiemelve, olvasatlan
}`}>

// 'Új' badge csak olvasatlan értesítéseknél:
{!notification.is_read && (
  <span className='badge bg-primary me-2'>Új</span>
)}

// Olvasottnak jelölő gomb szintén csak olvasatlanoknál:
{!notification.is_read && (
  <button onClick={() => handleMarkAsRead(notification.id)}>
    Olvasottnak jelölés
  </button>
)}
```

8. Hibakeresési napló

Hiba	Ok	Megoldás
Érvényes típusok: notice, announcement...	Frontend type értékek nem egyeztek a backend enum-mal	TYPE_OPTIONS frissítése a backend értékekre
A célcsoport megadása kötelező	target_role vs target mezőnév eltérés + hiányzó channel	Form Request rules()-ból pontos mezőneveket másolni
Badge nem jelenik meg	unread_count vs count mezőnév eltérés	unreadData?.count a helyes hivatkozás

– School Management App fejlesztési dokumentáció –